AuBase

Auction Marketplace Platform

Software Requirements Specification

User	Mohammad Shah & Waleed Bhatti
Date	April 15, 2026
Status	Draft
Prepared By	Development Team
System	AuBase — Full-Stack PHP/MySQL Auction Platform

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for AuBase (AuBase), a full-stack web-based auction marketplace. The document serves as the authoritative reference for all development, testing, and deployment activities.

1.2 Scope

AuBase is a PHP 8 + MySQL auction platform enabling users to list items for auction, place competitive bids, execute Buy It Now purchases, manage orders and shipments, and leave seller reviews. The system simulates a complete marketplace lifecycle from registration through payment release.

Core capabilities include:

- Item listing with categories, images (via Picsum placeholders), and configurable auction durations
- Real-time competitive bidding with concurrency-safe row-lock transactions
- Buy It Now fixed-price purchasing flow
- Order management including shipping tracking and delivery confirmation
- Seller review system with 1–5 star ratings
- Admin-controlled system time simulation for demo and testing

1.3 Definitions and Abbreviations

SRS	Software Requirements Specification
AuBase	AuBase auction platform
BIN	Buy It Now — fixed-price immediate purchase option
CSRF	Cross-Site Request Forgery — web security attack vector
DXA	Document unit (1/20 of a point; 1440 DXA = 1 inch)
CurrentTime	Single-row MySQL table used to simulate system clock for auction status
PHP	Hypertext Preprocessor — server-side scripting language
PDO/mysqli	PHP database access layers used for prepared statements
Seller	Authenticated user who lists items and sets auction parameters
Bidder/Buyer	Authenticated user who places bids or executes BIN purchases

1.4 Document Overview

This SRS is organized across five pages as follows:

1. Introduction — purpose, scope, definitions
2. Overall Description — product context, users, and assumptions
3. Functional Requirements — detailed feature requirements by module
4. Non-Functional Requirements — performance, security, reliability
5. Data Model and Constraints — schema summary and integrity rules

2. Overall Description

2.1 Product Perspective

AuBase is a self-contained web application with no external marketplace dependencies. It runs on a standard LAMP stack (Linux, Apache, MySQL, PHP) and is accessible via any modern web browser. The system uses a configurable CurrentTime table to allow simulation of auction open/closed states, making it suitable for both live deployment and controlled demonstration scenarios.

2.2 User Classes

Role	Access Level	Primary Activities
Guest	Read-only	Browse catalog, view item details and bid history (no bidding or listing)
Registered User	Authenticated	Bid on items, purchase BIN, manage bids and purchases, leave reviews
Seller	Authenticated	List items, set auction parameters, manage shipping options, receive payments
Admin	Full	Manage system time, import demo data, oversee all users and listings

2.3 Assumptions and Dependencies

- PHP 8.0 or higher is available on the server with the mysqli extension enabled.
- MySQL 5.7 or higher (or MariaDB equivalent) is accessible with create/read/write permissions.
- The application is deployed under a web server (Apache, Nginx, or PHP built-in server) with public/ as the document root.
- Email features (verification, password reset) require a configured SMTP server and AUBASE_MAIL_FROM environment variable; these are optional for local development.
- The CurrentTime table contains exactly one row (id = 1) and all auction status logic references this row as 'now'.
- Demo image thumbnails are served from picsum.photos using item_id as a seed; internet access is required for images to render.

2.4 End-to-End Flow

The standard lifecycle for an auction transaction proceeds as follows:

6. Seller adds bank info, creates a listing with at least one shipping option, and sets auction duration.
7. Buyers (with a valid credit card on file) place bids above the current price or execute a Buy It Now purchase.

8. When the auction closes (end_time < CurrentTime.system_time), the highest bidder becomes the winner.
9. Winner checks out: selects shipping option, confirms payment (simulated), generating an Order record.
10. Seller enters tracking number via their dashboard Orders to Ship view.
11. Buyer confirms delivery; payment is released to seller (simulated). Buyer may then leave a review.

3. Functional Requirements

3.1 Authentication

ID	Requirement	Priority
FR-A01	Users shall register with a unique username, unique email, and password. Password is stored using PHP password_hash.	**High**
FR-A02	Users shall log in with username/email and password; the system verifies with password_verify.	**High**
FR-A03	All session-protected pages shall redirect unauthenticated users to the login page.	**High**
FR-A04	Optional email verification flow shall prevent login until the verification link is clicked (when AUBASE_MAIL_FROM is configured).	**Medium**
FR-A05	Forgot-password flow shall send a time-limited token to the registered email for password reset.	**Medium**

3.2 Catalog and Search

ID	Requirement	Priority
FR-C01	The home catalog shall display paginated item listings with title, current price, bid count, and auction status badge (Open / Closed / BIN).	**High**
FR-C02	Users shall filter listings by category and search by keyword across item names.	**High**
FR-C03	Tabs for All, Open, Closed, and Buy It Now shall filter the catalog accordingly.	**High**
FR-C04	Catalog statistics (total items, total bids, live auctions) shall be displayed on the home page.	**Medium**

3.3 Bidding and Buy It Now

ID	Requirement	Priority
FR-B01	Authenticated buyers with a valid credit card on file shall be able to place bids on open auctions.	**High**
FR-B02	Each bid must be strictly greater than the current auction price; the system shall reject equal or lower bids.	**High**
FR-B03	Bidding shall use a row lock and database transaction to prevent race conditions on concurrent bids.	**High**

ID	Requirement	Priority
FR-B04	Sellers shall not be permitted to bid on their own listings.	High
FR-B05	When a Buy It Now price is set, eligible buyers may purchase immediately, closing the auction.	High
FR-B06	Bid history (bidder, amount, time) shall be visible on the item detail page.	Medium

3.4 Selling

ID	Requirement	Priority
FR-S01	Sellers must have a BankInfo record before creating a listing.	High
FR-S02	A listing shall include title, description, starting price, optional BIN price, category, location, country, and auction duration (1–14 days).	High
FR-S03	Every auction must have at least one ShippingOption (or pickup) added before the listing goes live.	High
FR-S04	Sellers shall view and remove their own active listings from the dashboard.	Medium

3.5 Order Management

ID	Requirement	Priority
FR-O01	After auction close, only the highest bidder shall be able to initiate checkout.	High
FR-O02	Checkout shall record a single Order per auction with payment status, selected shipping option, and card reference.	High
FR-O03	Seller dashboard shall show an 'Orders to Ship' view; sellers shall enter a tracking number.	High
FR-O04	Buyer dashboard shall show 'My Purchases'; buyers shall confirm delivery to release simulated payment.	High
FR-O05	After delivery confirmation, the buyer shall be able to leave one review per auction (rating 1–5 + comment).	Medium

4. Non-Functional Requirements

4.1 Security

ID	Requirement	Priority
NF-S01	All state-changing forms (bid, listing, account update, withdraw, delete) shall include and verify a CSRF token.	**High**
NF-S02	All database queries on untrusted input shall use prepared statements (mysqli or PDO).	**High**
NF-S03	Passwords shall never be stored in plaintext; PHP password_hash / password_verify are mandatory.	**High**
NF-S04	Session cookies shall be configured with HttpOnly and SameSite=Strict flags.	**High**

4.2 Performance

ID	Requirement	Priority
NF-P01	The catalog page shall load within 2 seconds for up to 10,000 items using indexed queries and pagination.	**Medium**
NF-P02	Bid submission shall complete within 1 second under normal load, including the row-lock transaction.	**High**
NF-P03	Database indexes shall be maintained on auction status fields, foreign keys, and search columns.	**Medium**

4.3 Reliability and Maintainability

ID	Requirement	Priority
NF-R01	The application shall run on PHP 8.0+ with no external front-end build tools (plain HTML/CSS/JS).	**High**
NF-R02	Database schema migrations shall be provided as discrete versioned SQL/PHP scripts.	**Medium**
NF-R03	All environment-specific settings (DB credentials, mail config) shall be stored in a .env file, not in source code.	**High**
NF-R04	The import script shall be executable from CLI or browser (with key) and idempotent on re-run.	**Low**

5. Data Model and Constraints

5.1 Entity Summary

Entity	Key Constraint	Description
User	`user_id PK`	Unique username and email; all participants (buyers, sellers, admins) are Users.
Item	`item_id PK`	Every item has a non-null name and a seller_id referencing User.
Category	`category_id PK`	Unique category names; linked to Items via Item_Category junction table.
Item_Category	`(item_id,category_id)`	Many-to-many; an item cannot belong to the same category twice.
Auction	`auction_id PK`	One auction per item (item_id UNIQUE). end_time > start_time enforced by CHECK.
Bid	`bid_id PK`	Tied to Auction and a bidder User. Each bid must exceed current_price (application logic).
CreditCard	`card_id PK`	Required for bidding/BIN. Validated (non-expired) at bid time (application logic).
ShippingOption	`shipping_option_id PK`	Tied to Auction. At least one required before listing goes live (application logic).
BankInfo	`seller_id UNIQUE`	One bank record per seller. Required before listing creation (application logic).
Order	`auction_id UNIQUE`	One order per auction. Payment status ENUM: pending / paid / failed.
Review	`review_id PK`	One per buyer per auction (application logic). Rating CHECK BETWEEN 1 AND 5.
CurrentTime	`id PK (row id=1)`	Single-row table. system_time is the application clock for all auction status decisions.

5.2 Schema-Enforced Constraints

- PRIMARY KEY uniqueness on all entities.
- FOREIGN KEY referential integrity across all relationships (CASCADE DELETE where appropriate).
- Auction.end_time > Auction.start_time — enforced by SQL CHECK constraint.
- Review.rating BETWEEN 1 AND 5 — enforced by SQL CHECK constraint.
- Auction.item_id UNIQUE — at most one auction per item.
- Order.auction_id UNIQUE — at most one order per auction.
- BankInfo.seller_id UNIQUE — at most one bank record per seller.
- Category.name UNIQUE, NOT NULL — no duplicate category names.

- Order.payment_status ENUM('pending','paid','failed') — restricted to defined values.

5.3 Application-Enforced Constraints

- Bid amount must strictly exceed Auction.current_price — validated in PHP with SELECT FOR UPDATE row lock inside a transaction.
- A seller cannot place bids on their own auction (seller_id check against session user_id).
- Bidder must have a non-expired CreditCard on file before placing a bid or executing BIN.
- Seller must have a BankInfo record before creating a listing.
- An auction must have at least one ShippingOption before going live.
- Only the highest bidder after auction close may initiate checkout.
- Only one review per buyer per auction (checked before insert).
- All auction open/closed status decisions use CurrentTime.system_time, not the server's real clock.

5.4 Revision History

Version	Date	Author	Description
1.0	April 15, 2026	Development Team	Initial release — covers full AuBase feature set as implemented.

6. Weekly Project Log

This log records development activity on AuBase by week, including tasks completed, challenges encountered, and total hours worked.

Week	Dates	Work Completed	Challenge	Hrs
Week 1	Jan 13–19	Project kickoff. Set up repo, configured LAMP stack, designed ER diagram, created base schema (db.sql) with User, Item, Auction, Bid, and Category tables.	*Choosing between PDO and mysqli; settled on mysqli with prepared statements for consistency.*	**12**
Week 2	Jan 20–26	Built register and login pages with password_hash/verify. Implemented session management, CSRF token helper, config.php with .env loading.	*CSRF token not persisting across tabs; fixed by storing in session rather than regenerating per-request.*	**14**
Week 3	Jan 27–Feb 2	Developed home catalog (index.php): pagination, category filter, keyword search, Open/Closed/BIN tabs, and stats counters. Applied navy/gold CSS theme.	*Pagination + filter query string conflicts; resolved with consistent URL parameter merging.*	**15**
Week 4	Feb 3–9	Built item detail page (item.php): bid history, minimum next bid, Buy It Now. Implemented bid submission with SELECT FOR UPDATE row lock in a transaction.	*Race condition on concurrent bids; row-lock transaction resolved duplicate bid amounts.*	**16**
Week 5	Feb 10–16	Created sell flow (sell.php): listing form, category picker, auction duration. Added ShippingOption creation and BankInfo check before listing goes live.	*Enforcing at least one shipping option required a two-step form flow before publish.*	**13**
Week 6	Feb 17–23	Built dashboard (dashboard.php): My Bids with withdraw, My Listings with remove, success toasts. Added CSRF on all dashboard actions.	*Withdraw logic needed to verify auction still open; edge case when CurrentTime changes mid-session.*	**14**
Week 7	Feb 24–Mar 2	Implemented checkout flow: winner detection, simulated payment, Order record creation. Added seller Orders to Ship view and tracking number entry.	*Reliably identifying the winner across CurrentTime simulation required careful end_time comparison.*	**15**
Week 8	Mar 3–9	Added delivery confirmation and simulated payment release. Built Review system:	*Preventing double reviews required an*	**12**

Week	Dates	Work Completed	Challenge	Hrs
		1–5 star form, one-review-per-auction enforcement, seller rating on item page.	*application-level unique check before insert.*	
Week 9	Mar 10–16	Built account settings: update profile, change username/email/password, activity stats, soft-delete with deleted_at migration.	*Soft delete required filtering deleted_at IS NOT NULL across all user-related queries site-wide.*	**13**
Week 10	Mar 17–23	Implemented optional email verification and forgot/reset password with time-limited tokens. Configured AUBASE_MAIL_FROM, tested with local Mailhog relay.	*Local SMTP setup for dev; documented Mailhog configuration in README for contributors.*	**11**
Week 11	Mar 24–30	Built demo data import (scripts/import.php): parsed items-0 to items-39 JSON, idempotent re-run, accessible via CLI or browser with AUBASE_IMPORT_KEY.	*Inconsistent JSON structure across eBay data files required defensive null-check parsing.*	**10**
Week 12	Mar 31–Apr 6	System testing: auction open/closed edge cases, bid concurrency, checkout end-to-end, review flow. Fixed several missing redirect-after-POST cases.	*Closed-auction BIN button remained clickable; fixed with server-side status check on submit.*	**14**
Week 13	Apr 7–13	Final polish: responsive CSS pass, Picsum placeholder images (seed by item_id), README writeup, migration scripts cleanup, inline code comments.	*Picsum seeds needed to be stable per item; using item_id directly as seed guaranteed consistency.*	**10**
Week 14	Apr 14–15	Wrote SRS documentation. Final submission prep: schema vs constraints doc review, end-to-end demo walkthrough, README screenshots section added.	*Ensuring SRS constraints matched db.sql exactly required careful side-by-side comparison.*	**6**
TOTAL	Jan 13 – Apr 15, 2026	Auth, catalog, bidding, selling, orders, reviews, account settings, import, documentation		**175**